

TaskHub — Milestones 1, 2 & 3

Interview Questions & Answers

Stack: React.js • React Native • Next.js • TypeScript • Node.js • Express • PostgreSQL • Prisma • Tailwind CSS • Material UI • Redux Toolkit • REST • JWT • OAuth • Jest • GitHub Actions • Sentry • Swagger/OpenAPI • ESLint • AWS • Firebase

Interview method: Answer in this order: Definition → Why we use it → TaskHub example → trade-off/alternative if relevant.

Milestone 1 — Project & Backend Foundation

1. What is TaskHub's overall architecture?

Answer: React web and React Native mobile clients communicate with a Node.js/Express REST API. The backend uses Prisma to access PostgreSQL, with AWS/Firebase and monitoring/testing/CI tools added around the core system.

2. What is Node.js?

Answer: Node.js is a JavaScript runtime based on Chrome's V8 engine that lets us execute JavaScript or TypeScript on the server.

3. Why use Express.js?

Answer: Express is a lightweight Node.js framework for HTTP servers and REST APIs. It provides routing, middleware, request/response handling, and error handling.

4. What is middleware?

Answer: Middleware is a function that runs during the Express request-response lifecycle. It can authenticate, validate, log, parse data, or handle errors.

5. What is CORS?

Answer: CORS controls which browser origins can access an API. It matters when the frontend and backend run on different origins.

6. Why use Helmet?

Answer: Helmet adds security-related HTTP headers to Express responses and helps establish safer browser defaults.

7. Why use dotenv?

Answer: dotenv loads configuration from environment variables, keeping values such as database URLs and secrets out of application source code.

8. Why use Zod?

Answer: Zod validates incoming data at runtime and works well with TypeScript. We can validate request bodies, parameters, queries, and configuration.

9. Why TypeScript?

Answer: TypeScript provides static typing, better IDE support, safer refactoring, and clearer contracts across the full-stack codebase.

10. What is REST?

Answer: REST is an architectural style for resource-oriented HTTP APIs. GET reads, POST creates, PATCH updates partially, and DELETE removes resources.

11. 401 vs 403?

Answer: 401 means authentication is missing or invalid. 403 means the request is understood but the authenticated user is not allowed to perform the operation.

12. What is ESLint?

Answer: ESLint statically analyzes JavaScript or TypeScript and flags problematic patterns and consistency issues before runtime.

13. Why GitHub Actions?

Answer: GitHub Actions automates CI/CD. TaskHub can run lint, tests, type checks, builds, and later deployment workflows on pushes or pull requests.

14. Why a monorepo?

Answer: A monorepo keeps web, mobile, backend, and shared packages in one repository, making shared types and coordinated changes easier to manage.

15. What is an HTTP status code 201?

Answer: 201 Created indicates that a request successfully created a new resource, commonly returned by a successful POST create operation.

Milestone 2 — PostgreSQL + Prisma

1. What is PostgreSQL?

Answer: PostgreSQL is an open-source relational database. It stores structured data in tables and supports relationships, constraints, indexes, and transactions.

2. Why PostgreSQL for TaskHub?

Answer: TaskHub has strongly related data such as users, projects, memberships, and tasks. A relational database provides strong integrity and transaction support.

3. What is Prisma?

Answer: Prisma is a TypeScript-friendly ORM/database toolkit that provides a schema, migrations, generated client, and type-safe queries.

4. What is an ORM?

Answer: Object-Relational Mapping lets application code work with database records through programming-language objects and methods rather than writing SQL for every operation.

5. PostgreSQL vs Prisma?

Answer: PostgreSQL is the database that stores the data. Prisma is the application-side data-access layer used by our TypeScript backend to communicate with it.

6. What is a primary key?

Answer: A primary key uniquely identifies a row. In TaskHub, entities use UUID string IDs.

7. What is a foreign key?

Answer: A foreign key references a key in another table and maintains a relationship, such as `Project.ownerId` referencing `User.id`.

8. What is one-to-many?

Answer: One record can relate to many records. For example, one user can own multiple projects.

9. What is many-to-many?

Answer: Many users can belong to many projects. TaskHub represents this with the `ProjectMember` junction table.

10. Why ProjectMember?

Answer: It implements the many-to-many relationship and also stores membership-specific information such as `OWNER`, `MANAGER`, or `MEMBER`.

11. What is a unique constraint?

Answer: A unique constraint prevents duplicate values. TaskHub makes user email unique to prevent duplicate registrations.

12. What is an index?

Answer: An index speeds up searches and filtering on selected columns. It improves reads but uses storage and adds some write overhead.

13. What is a migration?

Answer: A migration is a version-controlled database schema change that can be applied consistently across development, staging, and production.

14. prisma migrate vs prisma generate?

Answer: `migrate` creates/applies schema migrations; `generate` creates the Prisma Client code used by the application.

15. When use raw SQL?

Answer: Prisma is preferred for normal application queries, but raw SQL can be justified for complex queries, database-specific features, or cases requiring precise SQL control.

Milestone 3 — REST API + Backend Architecture

1. Why layered architecture?

Answer: Separating controllers, services, and repositories keeps responsibilities clear, improves testing, and makes the backend easier to maintain.

2. What does a Controller do?

Answer: It handles HTTP concerns: reads validated input, calls the appropriate service, and builds the HTTP response. Complex business logic should not live there.

3. What does a Service do?

Answer: A service contains business/application logic and coordinates repositories or other services without depending heavily on Express request/response objects.

4. What does a Repository do?

Answer: A repository abstracts database access, keeping Prisma queries and persistence details separate from business logic.

5. Why not put Prisma queries in controllers?

Answer: That makes controllers large and tightly coupled to the database. Repositories centralize data access and make services easier to test.

6. Explain the request flow.

Answer: Client → Express route → middleware → controller → service → repository → Prisma → PostgreSQL, then the result travels back to the client.

7. What is validation?

Answer: Validation verifies that incoming data follows the API contract before business logic executes. Zod is used for runtime schema validation.

8. Authentication vs authorization?

Answer: Authentication answers who the user is. Authorization answers what that user is allowed to do.

9. What is JWT?

Answer: JSON Web Token is a signed token containing claims that a server can verify. TaskHub will use JWTs for API authentication.

10. Why access and refresh tokens?

Answer: A short-lived access token limits exposure if stolen. A refresh token allows a client to obtain a new access token without frequent login.

11. What is centralized error handling?

Answer: A common Express error middleware converts known application errors into consistent responses and logs unexpected errors.

12. Why version APIs?

Answer: Versioning such as /api/v1 lets the backend evolve without immediately breaking existing web and mobile clients.

13. What is Swagger/OpenAPI?

Answer: OpenAPI is a standard API description format. Swagger tooling can render interactive documentation for developers and testers.

14. What is Jest?

Answer: Jest is a JavaScript/TypeScript testing framework used for unit tests and, with appropriate setup, integration tests.

15. What is Sentry?

Answer: Sentry monitors runtime errors and performance, providing stack traces and context that help diagnose production problems.

Rapid Revision

Term	Short interview answer
Express	Node.js framework for HTTP APIs, routing, and middleware.
Middleware	Function executed during the Express request-response lifecycle.
REST	Resource-oriented HTTP API style.
PostgreSQL	Relational database storing TaskHub data.
Prisma	Type-safe ORM/database toolkit.
Migration	Version-controlled database schema change.
JWT	Signed token used for verifiable authentication claims.
OAuth	Delegated authorization protocol used by external identity providers.
Zod	Runtime schema validation library for TypeScript.
Swagger/OpenAPI	Standard way to describe and document APIs.
Jest	Testing framework for JavaScript/TypeScript.
Sentry	Production error and performance monitoring.
Repository	Data-access layer.
Service	Business/application logic layer.
Controller	HTTP request/response layer.

Note: Milestone 3 is focused on the REST API and Controller → Service → Repository architecture that we build after the database foundation.